

Projeto AGROCONTROL

Sistema de Gestão do Agronegócio

Curso Técnico em Desenvolvimento de Sistemas – SENAI

1. Pesquisa do ramo de trabalho

1. Qual é o ramo escolhido?

Agronegócio.

2. Qual empresa ou tipo de negócio será utilizado como referência?

Uma propriedade rural fictícia de pequeno/médio porte, chamada Fazenda AgroVida, utilizada apenas como referência para o desenvolvimento acadêmico.

3. O que essa empresa faz?

A Fazenda AgroVida realiza atividades agrícolas, como planejamento de culturas, utilização de máquinas, controle de insumos e acompanhamento das atividades realizadas na propriedade.

4. Quem são seus clientes?

Os clientes são compradores da produção agrícola, como empresas, cooperativas, distribuidores e outros participantes da cadeia do agronegócio.

5. Quais são os principais serviços ou produtos oferecidos?

Produção agrícola, principalmente culturas de grãos, além da comercialização da produção obtida na propriedade.

6. Quais informações precisam ser cadastradas?

Produtores, funcionários, culturas, áreas plantadas, previsão de produção, máquinas, horas de uso das máquinas, insumos, quantidades em estoque, estoque mínimo, atividades agrícolas, datas e custos.

7. Quais problemas poderiam ser resolvidos por um sistema?

Dificuldade para organizar os cadastros, acompanhar o estoque de insumos, controlar máquinas, registrar atividades agrícolas e visualizar os custos da produção.

8. Como o processo é realizado atualmente?

Na situação-problema proposta, parte dos registros pode ser realizada manualmente, por anotações, planilhas ou informações separadas. Isso dificulta a centralização dos dados.

9. Quais dificuldades podem ocorrer nesse processo?

Perda de informações, dificuldade para localizar registros, falta de acompanhamento do estoque, esquecimento de atividades, dificuldade para calcular custos e pouca visão geral da propriedade.

10. Como um sistema poderia ajudar?

O sistema pode centralizar os cadastros, registrar atividades, acompanhar o estoque, identificar insumos abaixo do estoque mínimo, controlar máquinas e gerar um relatório resumido para apoiar a gestão.

2. Nome do projeto

AgroControl — Sistema de Gestão do Agronegócio.

3. Descrição do problema

A propriedade rural utilizada como referência possui diversas informações relacionadas à produção agrícola, máquinas, insumos, funcionários e atividades. Quando esses dados ficam espalhados em anotações ou controles separados, torna-se mais difícil acompanhar a situação da propriedade.

Problema identificado: como desenvolver um sistema de console em C# que centralize os principais dados da propriedade rural e facilite o controle de culturas, máquinas, insumos, atividades e custos?

4. Requisitos funcionais

RF01 — O sistema deverá permitir cadastrar produtores.

RF02 — O sistema deverá permitir cadastrar funcionários.

RF03 — O sistema deverá permitir cadastrar culturas.

RF04 — O sistema deverá permitir cadastrar máquinas.

RF05 — O sistema deverá permitir cadastrar insumos.

RF06 — O sistema deverá permitir registrar atividades agrícolas.

RF07 — O sistema deverá listar os produtores cadastrados.

RF08 — O sistema deverá listar as culturas cadastradas.

RF09 — O sistema deverá listar as máquinas cadastradas.

RF10 — O sistema deverá listar os insumos cadastrados.

RF11 — O sistema deverá listar as atividades agrícolas.

RF12 — O sistema deverá identificar insumos com estoque baixo.

RF13 — O sistema deverá gerar um relatório de gestão.

RF14 — O sistema deverá salvar os dados em arquivo JSON.

RF15 — O sistema deverá carregar os dados de um arquivo JSON.

5. Requisitos não funcionais

RNF01 — O sistema deverá ser desenvolvido em C#.

RNF02 — O projeto deverá ser executado no Visual Studio.

RNF03 — O sistema deverá possuir interface de console organizada.

RNF04 — O sistema deverá apresentar mensagens de erro para entradas inválidas.

RNF05 — O código deverá utilizar Programação Orientada a Objetos.

RNF06 — O código deverá ser organizado em classes, interface e serviços.

RNF07 — Os dados poderão ser persistidos em formato JSON.

RNF08 — O sistema deverá possuir menu de fácil utilização.

6. Regras de negócio

RN01 — Um produtor deve possuir nome, documento e propriedade.

RN02 — Um funcionário deve possuir nome, documento e cargo.

RN03 — Uma cultura deve possuir área maior que zero.

RN04 — A previsão de produção de uma cultura deve ser maior que zero.

RN05 — As horas de uso de uma máquina não podem ser negativas.

RN06 — A quantidade de um insumo não pode ser negativa.

RN07 — O estoque mínimo de um insumo não pode ser negativo.

RN08 — Um insumo deve gerar alerta quando sua quantidade for menor ou igual ao estoque mínimo.

RN09 — O custo de uma atividade agrícola não pode ser negativo.

RN10 — Uma atividade agrícola deve possuir data, descrição e cultura relacionada.

7. Identificação das classes

Pessoa (classe base)

Produtor

Funcionario

Cultura

Maquina

Insumo

AtividadeAgricola

DadosSistema

ArquivoService

RelatorioGestao

IRelatorio

8. Conceitos de POO utilizados

Classes

As informações do sistema são representadas por classes como Cultura, Maquina e Insumo.

Objetos

As classes são instanciadas no programa para representar produtores, culturas, máquinas e outros elementos.

Propriedades

As classes possuem propriedades para armazenar seus dados.

Métodos

Existem métodos como EstoqueBaixo(), RegistrarUso() e GerarRelatorio().

Construtores

Os objetos recebem seus dados iniciais por construtores.

Encapsulamento

Várias propriedades utilizam private set para impedir alterações externas indevidas.

this

É utilizado na classe RelatorioGestao para diferenciar campos da classe dos parâmetros recebidos.

static

GeradorId e ArquivoService utilizam membros estáticos.

Herança

Produtor e Funcionario herdam de Pessoa.

Polimorfismo

O método Descricao() é virtual em Pessoa e sobrescrito com override em Produtor e Funcionario.

Interface

A interface IRelatorio define o comportamento GerarRelatorio().

Exceções

O programa utiliza try/catch e validações para tratar entradas inválidas.

Serialização

Os dados são transformados em JSON e gravados em arquivo.

9. Organização do projeto

AgroControl

- └─ Program.cs
- └─ AgroControl.csproj
- └─ Classes
 - └─ Pessoa.cs
 - └─ Produtor.cs
 - └─ Funcionario.cs
 - └─ Cultura.cs
 - └─ Maquina.cs
 - └─ Insumo.cs
 - └─ AtividadeAgricola.cs
 - └─ DadosSistema.cs
- └─ Interfaces
 - └─ IRelatorio.cs
- └─ Services
 - └─ ArquivoService.cs
 - └─ RelatorioGestao.cs

10. Menu do sistema

- 1 - Cadastrar produtor
- 2 - Cadastrar funcionário
- 3 - Cadastrar cultura

- 4 - Cadastrar máquina
- 5 - Cadastrar insumo
- 6 - Registrar atividade agrícola
- 7 - Listar produtores
- 8 - Listar culturas
- 9 - Listar máquinas
- 10 - Listar estoque de insumos
- 11 - Listar atividades
- 12 - Relatório de gestão
- 13 - Salvar dados
- 14 - Carregar dados
- 0 - Sair

11. Funcionalidade surpresa

Nome: Alerta de estoque mínimo.

O sistema verifica automaticamente se a quantidade de um insumo está menor ou igual ao estoque mínimo cadastrado. Quando isso acontece, o sistema mostra um alerta na listagem de estoque e contabiliza a quantidade de alertas no relatório.

Problema resolvido: ajuda a identificar rapidamente possíveis necessidades de reposição de insumos.

Conceito de POO utilizado: encapsulamento e métodos, especialmente o método `EstoqueBaixo()` da classe `Insumo`.

Regra de negócio: RN08 — um insumo deve gerar alerta quando sua quantidade for menor ou igual ao estoque mínimo.

12. Desafio extra 02 — Se eu fosse o cliente

Melhoria escolhida: relatório de gestão da propriedade.

O relatório reúne quantidade de produtores, funcionários, culturas, área total, produção prevista, máquinas, insumos, alertas de estoque, atividades e custo total das atividades. Dessa forma, o usuário consegue visualizar um resumo da situação cadastrada sem precisar consultar cada menu separadamente.

13. Desafio Master

Funcionalidade criada: controle de custos das atividades agrícolas.

Problema que resolve: permite registrar quanto cada atividade agrícola custou e calcular o custo total das atividades cadastradas.

Regra de negócio: o custo não pode ser negativo.

Classes envolvidas: AtividadeAgricola e RelatorioGestao.

Conceitos de POO: classe, propriedade, construtor, encapsulamento, método e interface.

14. Plano de testes

Teste	Procedimento	Resultado esperado	Status
Teste 1	Cadastrar informações válidas.	Cadastro deve ser realizado e aparecer na listagem.	Aprovado
Teste 2	Cadastrar informação inválida, como quantidade negativa.	Sistema deve impedir o cadastro e apresentar erro.	Aprovado
Teste 3	Pesquisar/consultar informação existente por meio das listagens.	Registro deve aparecer na lista.	Aprovado
Teste 4	Consultar quando não existem dados cadastrados.	Sistema deve informar que não há registros.	Aprovado
Teste 5	Executar relatório sem dados.	Relatório deve ser gerado com valores zerados.	Aprovado
Teste 6	Salvar dados.	Arquivo JSON deve ser criado.	Aprovado
Teste 7	Fechar e abrir o sistema.	Programa deve iniciar normalmente.	Aprovado
Teste 8	Carregar dados salvos.	Dados do JSON devem retornar às listas.	Aprovado

15. Como executar no Visual Studio

Abrir o arquivo AgroControl.csproj no Visual Studio.

Confirmar que o projeto utiliza .NET 8 ou compatível.

Pressionar F5 ou Ctrl+F5.

Usar o menu do console para cadastrar os dados.

Testar entradas válidas e inválidas.

Usar a opção 13 para salvar.

Fechar e executar novamente.

Usar a opção 14 para carregar o arquivo JSON.

Testar o relatório e o alerta de estoque.

16. Conclusão

O projeto AgroControl apresenta uma solução de software para organizar informações básicas de uma propriedade rural. A aplicação foi estruturada em C# com Programação Orientada a Objetos e contempla cadastro, controle de estoque, registro de atividades, relatório e persistência em JSON. O projeto também demonstra conceitos de POO estudados durante o curso e possui uma funcionalidade adicional de alerta de estoque mínimo.

